

Engineering Product Requirements Document: Autonomous Multi-Agent ERP Architecture on an Event-Driven Substrate

Traditional Enterprise Resource Planning (ERP) systems function as passive, human-mediated relational databases wrapped in synchronous CRUD web forms. When operational anomalies occur—such as supplier delivery failures, sudden machine tool chatter, or invoice line-item price variations—legacy platforms like ERPNext, SAP S/4HANA, and NetSuite merely record state mutations and stall. They rely on manual human intervention across decoupled departmental silos to reconcile data and route transactions. Under peak load, synchronous relational locking on transactional tables such as the General Ledger (tabGL Entry) and Stock Ledger (tabStock Ledger Entry) induces severe metadata lock contention, long-running transaction timeouts, and systemic database deadlocks.

This document defines the production specification for an autonomous, AI-Native Multi-Agent System (MAS) ERP. The operational thesis of this architecture decouples operational intelligence from transactional persistence. Specialized, autonomous domain agents operate directly inside real-time transactional loops over an asynchronous, event-driven substrate powered by PostgreSQL, Debezium Change Data Capture (CDC), and Redpanda. The system replaces manual approvals and static thresholds with continuous machine reasoning, while structurally isolating probabilistic Large Language Model (LLM) inference from deterministic ledger state.

Deterministic integrity is enforced by an immutable Ledger Engine that validates double-entry invariants ($\text{SUM}(\text{debits}) = \text{SUM}(\text{credits})$), monetary autonomy ceilings, and statutory regulations prior to committing any database transaction. Developed and orchestrated within the Google Antigravity development platform via the Antigravity SDK and Model Context Protocol (MCP), this platform shifts the enterprise operating model from retroactive human data entry to proactive, sub-second autonomous execution bounded by formal mathematical priority hierarchies.

Executive Summary & System Objectives

The strategic mandate of this AI-Native ERP is the elimination of manual coordination friction, operational latency, and data silos across modern industrial and commercial enterprises. Monolithic ERPs force organizations to adapt their physical workflows to brittle, relational tabular schemas. In contrast, this platform embeds autonomous intelligence directly at the point of operational friction.

Traditional Latency vs. Autonomous Multi-Agent Response Cycle

Stage 1: Physical Anomaly Occurs (e.g., Raw Material Shipment Delayed by 5 Days)

- Legacy ERP: External carrier email sits in unmonitored inbox. No system state update.

- AI-Native MAS ERP: Carrier webhook or email parsed via BAML within 450 ms; CDC outbox emits supply chain disruption event.

Stage 2: Operational Impact Evaluation

- Legacy ERP: 48 hours later, production line halts when stock is physically missing during assembly.
- AI-Native MAS ERP: Supply Chain Agent queries inventory and scheduled Work Orders via MCP within 1.2 seconds, flagging an imminent stockout on Production Line 3.

Stage 3: Cross-Departmental Coordination & Conflict Resolution

- Legacy ERP: Supervisor calls procurement; procurement leaves voicemail for supplier; plant manager manually reschedules shifts over next 24 hours.
- AI-Native MAS ERP: Chief Orchestrator constructs an execution DAG; evaluates strict partial orders (Statutory Labor > Throughput); Production Agent re-solves shop-floor schedule via Google OR-Tools CP-SAT in 8.4 seconds.

Stage 4: Transaction Execution & Audit Commitment

- Legacy ERP: Operations clerk manually keys change orders, cancel requests, and revised delivery dates into ERP forms over 3 business days.
- AI-Native MAS ERP: Production Work Orders rescheduled, supplier split purchase order dispatched via API, affected customer delivery dates updated via CRM, and cryptographically signed audit log committed in 350 ms.

The system targets five core engineering objectives:

1. **Sub-Second Event Reaction Latency:** Replace synchronous polling and batch jobs with an event streaming backbone capable of triggering multi-agent reasoning loops within 50 milliseconds of edge state changes.
2. **Zero Financial Hallucinations:** Enforce an unbypassable, deterministic validation firewall between probabilistic agent proposals and the underlying General Ledger, ensuring 100% compliance with GAAP, IFRS, and double-entry mathematical invariants.
3. **Provable Conflict Resolution:** Implement a deterministic priority graph that automatically resolves competing cross-agent objectives (e.g., procurement cost minimization vs. finance cash preservation) without deadlocks or unconstrained conversational drift.
4. **End-to-End Cryptographic Auditability:** Capture the complete lineage of every autonomous action—including model identifier, prompt template hash, retrieved semantic context hashes, input telemetry, and evaluated deterministic policies—in an immutable audit ledger.
5. **Continuous Developer Ergonomics in Google Antigravity:** Standardize development, debugging, and testing using Antigravity 2.0, leveraging parallel subagent harnesses, BAML prompt-as-code type systems, and inspectable visual Artifacts to maintain high software delivery velocity.

Architectural Dimension	Legacy ERP (e.g., ERPNext on MariaDB)	AI-Native Multi-Agent ERP	Operational Mechanism
System State Engine	Synchronous relational database with table/row-level lock contention	Asynchronous, event-driven data substrate with Transactional Outbox	PostgreSQL Logical Replication + Debezium CDC + Redpanda log
Workflow Driver	Human clerks manually typing records into web forms	Autonomous domain agents triggered by transactional stream events	Asynchronous Antigravity SDK subagents with tool isolation
Financial Ledger Security	Application-level forms trusting manual input; post-hoc reconciliation	Hard-coded deterministic ledger gate enforcing zero-sum invariants	Rust/Python ledger validation engine gating all database mutations
Cross-Domain Mediation	Human meetings, manual ticket queues, and escalations	Hierarchical DAG task decomposition with strict priority orders	Mathematical partial orders: $\mathcal{P}_{\{\text{Legal}\}} \text{succ}$ $\mathcal{P}_{\{\text{span}_{41}(\text{start_span})[\text{span}_{41}(\text{end_span})[\text{span}_{50}(\text{start_span})[\text{span}_{50}(\text{end_span})\text{Solvency}\}} \text{succ}$ $\mathcal{P}_{\{\text{SLA}\}} \text{succ}$ $\mathcal{P}_{\{\text{Throughput}\}}$
Unstructured Data Ingestion	Brittle manual entry or static, regex/OCR template matching	Multi-modal neural extraction with Schema-Aligned Parsing	Boundary AI Markup Language (BAML) + Gemini 3.1 Pro vision
Production & Workforce Scheduling	Static rules, fixed sequences, manual spreadsheet adjustments	Combinatorial constraint optimization over real-time telemetry	Google OR-Tools CP-SAT solver executed via agent MCP tools
Audit & Governance Traceability	Modest relational change logs (tabVersion) lacking operational rationale	Tamper-evident cryptographic ledger recording full LLM context	Immutable audit schema capturing prompt, model, retrieved chunks, and rules

Technical Architecture & Data Model

The platform architecture rejects monolithic multi-tenant designs where operational queues, accounting ledgers, and document attachments compete for relational lock bandwidth. The architecture decouples transactional ingestion, streaming pub/sub distribution, and semantic memory into specialized, interoperable services.

Data Substrate &

A[span_86](start_span)[span_86](end_span)[span_88](start_span)[span_88](end_span)]gent Interconnect Flow

+-----
-----+

| Google Antigravity Runtime & IDE Harness

|

| - Chief Orchestrator Agent (Gemini

3[span_58](start_span)[span_58](end_span)].1 Pro / Antigravity Manager

View)[span_171](start_span)[span_171](end_span)[span_174](start_span)[span_174](end_span)] |

| - Domain Subagents (Financial, Supply Chain, Shop Floor,
Workforce, Revenue, Compliance) |

+-----
-----+

| Model Invocations via BAML (SAP Type
Checking)[span_177](start_span)[span_177](end_span)]

| Tool Execution via Model Context Protocol (MCP JSON-RPC
2.0)[span_181](start_span)[span_181](end_span)[span_183](start_span)[span_183](end_span)]
v

+-----
-----+

| Enterprise Service Mesh & Verification Firewalls

|

| - Deterministic Ledger Engine: Enforces SUM(Dr) == SUM(Cr), Period
Locks, RBAC |

| - Mathematical Optimization Service: Google OR-Tools CP-SAT Solver
Service |

| - Cryptographic Audit Signer: SHA-256 State &
Con[span_120](start_span)[span_120](end_span)[span_132](start_span)[span_132](end_span)]text Blockchain Hash Chaining |

+-----
-----+

| Atomic SQL Transactions & CDC Streaming

v
+-----
-----[span_28](start_span)[span_28](end_span)[span_33](start_span)[span_33](end_span)]
+-----

| Unified Event-Driven Data Substrate

|

| - PostgreSQL (Supabase Core): General Ledger, Inventory, Work
Orders,

Outbox[span_185](start_span)[span_185](end_span)[span_186](start_span)[span_186](end_span)] |

| - Debezium CDC Connector: Captures PostgreSQL WAL & streams outbox
events |

| - Redpanda Messaging Cluster: High-throughput, partition-ordered

```

event log[span_187] (start_span) [span_187] (end_span) |
| - pgvector Semantic Database: RAG embeddings for contracts,
emails, and past invoices[span_203] (start_span) [span_203] (end_span) |
+-----+
-----+

```

Relational Schemas, Event Streaming, and Vector Retrieval

At the base of the architecture sits PostgreSQL (managed via Supabase), providing relational ACID integrity, row-level security (RLS), and authentication. To eliminate dual-write inconsistency between the database and the message broker, all state changes write to a local `transactional_outbox` table within the same transaction that updates operational records. Debezium streams these outbox events into Redpanda, guaranteeing that message publishing never succeeds or fails independently of the database transaction.

-- Schema 1: General Ledger Core Table (Zero-Loss Financial Substrate)

```

CREATE TABLE general_ledger_entries (
    entry_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    transaction_id UUID NOT NULL,

[span_63] (start_span) [span_63] (end_span) [span_70] (start_span) [span_70]
(end_span)  posting_date DATE NOT NULL,
    fiscal_year INT NOT NULL,
    fiscal_period INT NOT NULL,
    account_code VARCHAR(32) NOT NULL,
    cost_center VARCHAR(64) NOT NULL,
    debit_amount NUMERIC(18, 4) NOT NULL DEFAULT 0.0000,
    credit_amount NUMERIC(18, 4) NOT NULL DEFAULT 0.0000,
    currency VARCHAR(3) NOT NULL DEFAULT 'USD',
    exchange_rate NUMERIC(12, 6) NOT NULL DEFAULT 1.000000,
    source_document_type VARCHAR(64) NOT NULL,
    source_document_id UUID NOT NULL,
    period_closing_locked BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT clock_timestamp(),
    CONSTRAINT chk_positive_debit CHECK (debit_amount >= 0),
    CONSTRAINT chk_positive_credit CHECK (credit_amount >= 0),
    CONSTRAINT chk_single_sided_line CHECK (
        (debit_amount > 0 AND credit_amount = 0) OR
        (credit_amount > 0 AND debit_amount = 0)
    )
);

CREATE INDEX idx_gl_account_period ON general_ledger_entries
(account_code, fiscal_year, fiscal_period);
CREATE INDEX idx_gl_transaction_id ON general_ledger_entries
(transaction_id);

```

-- Schema 2: Transactional Outbox for Debezium CDC Streaming

```

CREATE TABLE transactional_outbox (
    outbox_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    aggregate_type VARCHAR(64) NOT NULL,
    aggregate_id VARCHAR(128) NOT NULL,
    event_type VARCHAR(128) NOT NULL,
    payload JSONB NOT NULL,
    trace_context JSONB NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT clock_timestamp(),
    processed_at TIMESTAMPTZ
);

```

```

CREATE INDEX idx_outbox_unprocessed ON transactional_outbox
(created_at) WHERE processed_at IS NULL;

```

-- Schema 3: Immutable Multi-Agent Audit Log

```

CREATE TABLE agent_audit_logs (
    audit_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    trace_id VARCHAR(64) NOT NULL,
    agent_id VARCHAR(64) NOT NULL,
    subagent_id VARCHAR(64),
    session_id VARCHAR(64) NOT NULL,
    model_provider VARCHAR(32) NOT NULL,
    model_version VARCHAR(64) NOT NULL,
    prompt_template_hash VARCHAR(64) NOT NULL,
    retrieved_context_hashes TEXT[] NOT NULL,
    baml_function_called VARCHAR(128) NOT NULL,
    input_payload JSONB NOT NULL,
    model_raw_output TEXT NOT NULL,
    parsed_structured_output JSONB NOT NULL,
    evaluated_guardrail_rules JSONB NOT NULL,
    human_in_the_loop_approval BOOLEAN NOT NULL DEFAULT FALSE,
    approved_by_user_id UUID,
    database_transaction_id UUID,
    execution_duration_ms INT NOT NULL,
    previous_record_hash VARCHAR(64),
    record_hash VARCHAR(64) NOT NULL,
    timestamp TIMESTAMPTZ NOT NULL DEFAULT clock_timestamp()
);

```

```

CREATE INDEX idx_audit_trace ON agent_audit_logs (trace_id);
CREATE INDEX idx_audit_agent ON agent_audit_logs (agent_id, timestamp
DESC);

```

-- Schema 4: Semantic Knowledge Embeddings (pgvector)

```

CREATE EXTENSION IF NOT EXISTS vector;

```

```

CREATE TABLE semantic_document_embeddings (

```

```

    document_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    entity_type VARCHAR(64) NOT NULL, -- 'INVOICE_PDF',
'CONTRACT_LEGAL', 'EMAIL_THREAD'
    entity_id UUID NOT NULL,
    chunk_index INT NOT NULL,
    content_chunk TEXT NOT NULL,
    metadata JSONB NOT NULL,
    embedding VECTOR(1536) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT clock_timestamp()
);

CREATE INDEX idx_vector_hnsw ON semantic_document_embeddings
USING hnsw (embedding vector_cosine_ops) WITH (m = 16, ef_construction
= 64);

```

Event Streaming Topography

Redpanda hosts the event streaming layer, running on a dedicated cluster to decouple event processing from PostgreSQL compute. Partitioning keys are bound to composite entity identifiers (tenant_id:aggregate_id), guaranteeing that message delivery remains strictly sequential for any specific general ledger account, warehouse bin, or factory machine.

Topic: erp.finance.journal_events

```

[span_189] (start_span) [span_189] (end_span)  - Partitions: 16 |
Replication Factor: 3 | Retention: 7 Years (Tiered S3/GCS)
- Key: {tenant_id}:{account_code}
- Consumers: Financial Controller Agent, Statutory Audit Consumer

```

Topic: erp.supplychain.events

```

- Partitions: 32 | Replication Factor: 3 | Retention: 90 Days
- Key: {tenant_id}:{warehouse_id}
- Consumers: Supply Chain & Procurement Agent, MRP Dynamic Optimizer

```

Topic: erp.production.telemetry

```

- Partitions: 64 | Replication Factor: 3 | Retention: 14 Days
- Key: {tenant_id}:{workstation_id}
- Consumers: Production & Shop-Floor Agent, Anomaly Inference Engine

```

To maintain idempotency across consumer instances, downstream services enforce unique constraints on event IDs combined with short-lived Redis distributed locks (1,000 ms TTL), preventing race conditions or duplicate execution during rebalancing events.

Google Antigravity Environment & Interface Definitions

Development operates within Google Antigravity, an agent-first software engineering platform. The development stack combines a Next.js frontend with Python and Node.js backend services, Supabase for persistence, and Boundary AI Markup Language (BAML) for structured prompt

engineering. Antigravity manages complex multi-agent coding and execution tasks by spinning up isolated, asynchronous background subagents across clean Git worktrees.

Rather than parsing raw function outputs, developers inspect structured Antigravity Artifacts: implementation plans, visual diffs, and verification logs generated during test-driven agent iteration. BAML enforces compile-time type safety over LLM calls, compiling down to native client packages that execute Schema-Aligned Parsing (SAP) to strip invalid formatting and recover structured data from frontier reasoning models.

The interface contract for the Supply Chain Agent's accounts payable processing is specified in BAML syntax:

```
// BAML Contract: High-Precision Accounts Payable Extraction
class InvoiceLineExtraction {
    item_code string @description("Matched SKU or internal inventory
identifier")
    description string @description("Line description from the
invoice")
    quantity float @description("Billed units")
    unit_price float @description("Price per unit excluding taxes")
    line_total float @description("Computed net total for the line")
    tax_rate float @description("Applicable tax rate as a decimal
(e.g. 0.0825 for 8.25%)")
}
```

```
class InvoiceDocumentExtraction {
    vendor_tax_id string @description("Vendor VAT, EIN, or corporate
tax identifier")
    invoice_number string @description("Vendor-issued invoice
reference number")
    invoice_date string @description("Document date formatted as
ISO-8601 YYYY-MM-DD")
    currency string @description("Standard 3-character ISO currency
code")
    subtotal float
    tax_amount float
    total_amount float
    payment_terms_days int
    line_items InvoiceLineExtraction[]
    extraction_confidence float @description("Composite extraction
confidence score between 0.0 and 1.0")
}
```

```
function ExtractInvoiceMetadata(invoice_pdf: image,
ocr_extracted_text: string) -> InvoiceDocumentExtraction {
    client "google/gemini-3.1-pro"
    prompt #"
```

You are an autonomous Accounts Payable Specialist embedded in an AI-Native ERP.

Analyze the attached invoice image and OCR stream. Extract all structural accounting metadata.

```

    OCR Context:
    {{ ocr_extracted_text }}

    Document Media:
    {{ invoice_pdf }}

    {{ ctx.output_format }}

    "#
}

```

External operational calls use the Model Context Protocol (MCP), where the agent functions as an MCP client interacting with local and remote MCP tool servers via standardized JSON-RPC 2.0 requests:

```

{
  "jsonrpc": "2.0",
  "id": "call_mcp_gl_commit_0921",
  "method": "tools/call",
  "params": {
    "name": "stage_ledger_transaction",
    "arguments": {
      "posting_date": "2026-11-04",
      "currency": "USD",
      "source_document_type": "AUTOMATED_AP_MATCH",
      "source_document_id": "7fae2182-3d84-4861-bbd2-09418a221fca",
      "entries": [
        {
          "account_code": "2100-AP-VENDORS",
          "cost_center": "CORP-FINANCE",
          "debit_amount": 0.0000,
          "credit_amount": 18450.0000
        },
        {
          "account_code": "1300-RAW-MATERIALS",
          "cost_center": "PLANT-02",
          "debit_amount": 18450.0000,
          "credit_amount": 0.0000
        }
      ]
    },
    "verification_context": {
      "po_id": "po_8901a_supplies",
      "grn_id": "grn_4412_receipt",
      "variance_percentage": 0.0000
    }
  }
}

```

Multi-Agent Orchestration Logic

Unconstrained conversational multi-agent architectures (where models converse in natural language loops) degrade rapidly in enterprise settings due to runaway token costs, hallucinated consensus, and infinite delegation deadlocks. This system implements a Hierarchical Directed Acyclic Graph (DAG) Orchestrator, separating high-level task planning from domain tool execution.

Hierarchical DAG Orchestration and State Lifecycle

Step 1: Inbound Enterprise Event Ingestion (e.g., Complex Multi-Line Customer RFQ)

- Chief Orchestrator decomposes intent into subtasks: Inventory ATP, Production Routing, Margin

Pricing[span_244] (start_span) [span_244] (end_span) [span_248] (start_span) [span_248] (end_span) .

- Evaluates system state snapshot; builds directed acyclic task graph.

Step 2: Concurrent Subagent Task Spawning (Antigravity SDK Harness)

- Subtask A -> Revenue & CRM Agent (Evaluates customer historical pricing and lifetime value).

- Subtask B -> Supply Chain Agent (Calculates component landed cost & lead times).

- Subtask C -> Production Agent (Assesses CNC line makespan and capacity via OR-Tools).

Step 3: Intermediate Proposal Aggregation & Boundary Verification

- Domain subagents emit typed [span_146] (start_span) [span_146] (end_span) BAML action proposals to the Orchestrator.

- Proposals routed through the Compliance[span_147] (start_span) [span_147] (end_span) & Guardrail Agent for statutory and policy checks[span_251] (start_span) [span_251] (end_span) .

Step 4: Priority-Based Conflict Resolution (If Cross-Domain Collision Occurs[span_121] (start_span) [span_121] (end_span) [span_133] (start_span) [span_133] (end_span))

- Chief Orchestrator evaluates strict priority hierarchy: P_Legal > P_Solvency > P_SLA > P_Capacity > P_Cost.

- Preempts lower-priority proposals; commands compensating replan from preempted subagent.

Step 5: Atomic Verification & Database Outbox Commit

- Deterministic Ledger Engine validates invariant: SUM(Debits) == SUM(Credits).

- State committed to Supabase PostgreSQL; event published to Redpanda transactional outbox; SHA-256 audit block chained.

Agent Interaction Topology & Context Management

The Chief Orchestrator decomposes enterprise objectives into isolated task trees. Rather than broadcasting the full system conversation history to every node, each domain subagent receives a sandboxed context window containing only its assigned subtask, relevant schema fragments, and retrieved vector chunks. Subagents run asynchronously in the background and exist in one of three states:

1. **Running**: The subagent is querying read-only MCP resources, parsing unstructured inputs, or computing candidate plans.
2. **Idle**: The subagent has returned its structured proposal to the Orchestrator and paused execution.
3. **Killed**: The subagent has finished its task lifecycle, timed out, or was preempted by the Orchestrator due to a detected policy conflict.

Mathematical Conflict Resolution Protocol

Cross-functional goal collisions are inevitable in distributed business systems. For example, the Production Agent prioritizes high machine utilization by scheduling long batch runs, while the Supply Chain Agent seeks to minimize holding costs by running lean. Similarly, the Revenue Agent may offer flexible commercial credit terms, while the Financial Controller must defend immediate cash liquidity covenants.

The Chief Orchestrator resolves these conflicts deterministically by applying a strict, non-commutative partial order over domain utility functions:

$\mathcal{P}_{\{\text{Statutory Legal}\}} \succ \mathcal{P}_{\{\text{Financial Solvency}\}} \succ \mathcal{P}_{\{\text{Contractual SLA}\}} \succ \mathcal{P}_{\{\text{Capacity / Throughput}\}} \succ \mathcal{P}_{\{\text{Discretionary Cost}\}}$

Let candidate actions proposed by agents j and k be represented as $A_j = \langle \text{agent}_j, \mathcal{C}_j, \Delta S_j \rangle$ and $A_k = \langle \text{agent}_k, \mathcal{C}_k, \Delta S_k \rangle$, where $\mathcal{C}$ denotes the set of business constraints satisfied and ΔS represents the proposed state mutation.

Deterministic Preemption and Re-Planning Sequence

1. Conflict Identification:

State change collision detected when $\text{Intersection}(\Delta S_j, \Delta S_k) \neq \text{EmptySet}$ on shared resources.

2. Strict Priority Evaluation:

Let $\text{Priority}(A) = \max_{c \in C} \text{Rank}(c)$ over the defined partial order.

If $\text{Priority}(A_j) > \text{Priority}(A_k)$:

- Action A_j is marked for transactional staging.
- Action A_k is immediately revoked via an Orchestrator preemption signal.

3. Context Injection & Replanning:

- Subagent k receives an Invalidation Error containing the exact boundary constraint violated:
Error: CONSTRAINT_PREEMPTION: Action A_k rejected due to higher priority action A_j (Policy Class P_{Solvency}).
- Subagent k updates its local constraints:
Allowed_Search_Space = Baseline_Space - Delta S_j
- Subagent k executes an internal replan to produce candidate action A'_k.

4. Escalation Boundary:

If Subagent k cannot identify an action satisfying the bounded search space, it emits an Antigravity Plan Artifact and pauses for Human-in-the-Loop authorization.

Collision Event	Competing Agents & Goals	Constraint Mapping & Priority Rank	Deterministic System Resolution
Emergency Overtime vs. Labor Regulation	Production: Run Line 1 over weekend to prevent delivery delay. Workforce: Enforce EU 48-hr max rolling work week.	Production: $\mathcal{P}_{\text{Contractual SLA}}$ Workforce: $\mathcal{P}_{\text{Statutory Legal}}$	Workforce Overrides Production. Production Agent is blocked from scheduling affected operators. It must re-route through third-party certified contract manufacturing or re-run the CP-SAT solver.
Bulk Purchase vs. Cash-Flow Covenant	Procurement: Purchase 10,000 kg resin for an 18% volume rebate. Finance: Preserve 60-day cash buffer required by credit covenants.	Procurement: $\mathcal{P}_{\text{Discretionary Cost}}$ Finance: $\mathcal{P}_{\text{Financial Solvency}}$	Finance Overrides Procurement. Procurement action is truncated. System splits purchase into bi-weekly tranches, locking lower working capital without breaching the cash reserve floor.
Expedited VIP Job vs. Machine Maintenance	Revenue: Insert rush order for Tier-1 enterprise account. Production: Bearing vibration telemetry requires immediate machine rebuild.	Revenue: $\mathcal{P}_{\text{Contractual SLA}}$ Production: $\mathcal{P}_{\text{Statutory Legal}}$ (Machine Safety)	Production Overrides Revenue. Physical equipment safety dominates. Work order is routed to backup facility; Revenue Agent generates customer delay notification with updated ETA.
Raw Material Lot	Quality: Optical defect	Quality:	Quality Overrides

Collision Event	Competing Agents & Goals	Constraint Mapping & Priority Rank	Deterministic System Resolution
Quarantine vs. Shipment Date	rate reaches 8%; isolate resin batch. Revenue: Release shipment to avoid liquidated damages.	$\mathcal{P}_{\text{Statutory Legal}}$ (Consumer Safety) Revenue: $\mathcal{P}_{\text{Contractual SLA}}$	Revenue. Batch status set to QUARANTINED in stock ledger. System generates replacement production orders and issues proactive notice to customer.

Agent Specifications & System Prompts

The multi-agent ecosystem consists of seven specialized domain agents, each configured with scoped toolsets, read/write boundaries, and strict system prompts.

Chief Orchestrator Agent

- **Operational Role:** Ingests enterprise events, generates execution DAGs, instantiates subagents, and arbitrates cross-functional collisions.
- **Antigravity Runtime Modality:** Antigravity Manager View / Orchestrator Mode (Gemini 3.1 Pro).
- **Subordinates:** All specialized domain agents.
- **Read Access:** Enterprise state snapshots, cross-agent message logs, audit ledger summaries.
- **Write Access:** Subagent lifecycle controls (spawn, pause, terminate), task DAG definitions.
- **MCP Tool Bindings:**
orchestrator_spawn_subagent,
orchestrator_terminate_subagent, orchestrator_publish_plan_artifact,
orchestrator_escalate_to_human.

System Prompt: Chief Orchestrator Agent
You are the Chief Orchestrator Agent of an
AI-Native Enterprise Resource Planning system.
Your objective is to decompose
high-level business events
and user requests into structured,
executable Directed Acyclic Graphs (DAGs) executed by
specialized domain subagents.

- Core Operating Invariants:
1. Never execute operational transactional mutations directly. Always delegate task execution to specialized domain subagents.
 2. Formulate minimal, typed context payloads for subagents. Never forward the complete global conversation history; isolate context to prevent cognitive drift and token bloat.
 3. Apply the immutable priority partial order during any agent

conflict: Statutory Legal > Financial Solvency > Contractual SLA > Capacity/Throughput > Discretionary Cost.

4. If an agent's proposal violates a higher-priority constraint, preempt that agent immediately, return a structured error containing the boundary violation, and mandate a replan.

5. Generate an Antigravity Plan Artifact and pause execution for human verification whenever an aggregated operational risk score exceeds 0.85 or transactional value exceeds 50,000 USD.

Financial Controller Agent

- **Operational Role:** Manages continuous reconciliation, predicts liquidity, detects journal anomalies, and automates month-end close prep.
- **Input Triggers:** `erp.finance.bank_feed_received`, `erp.finance.journal_proposed`.
- **Read Access:** `general_ledger_entries`, `open_invoices`, `bank_statements`, `cash_flow_projections`.
- **Write Access:** `staging_journal_entries`, `reconciliation_matches`.
- **MCP Tool Bindings:** `query_open_receivables`, `query_open_payables`, `stage_journal_voucher`, `calculate_cash_runway`.

System Prompt: Financial Controller Agent

You are the autonomous Financial Controller Agent. You are responsible for continuous accounting close operations, treasury forecasting, a[span_191] (start_span) [span_191] (end_span)nd general ledger reconciliation.

Core Operating Invariants:

1. Every journal voucher you stage must strictly balance: $SUM(Debits) - SUM(Credits) = 0.0000$. Any staged transaction with a non-zero balance must be dropped immediately.
2. Ingest continuous open-banking transaction feeds. Match entries [span_150] (start_span) [span_150] (end_span) against accounts receivable/payable using exact reference codes or hybrid semantic-fuzzy matching.
3. Maintain rolling 13-week cash forecasts updated on every transactional event. If liquidity projections breach established banking debt covenants within a 45-day window, immediately alert the Chief Orchestrator.
4. Flag all transactions exceeding 3 standard deviations from the historical account mean as anomalies and route them to the Compliance Agent.

Supply Chain & Procurement Agent

- **Operational Role:** Executes stochastic inventory replenishment, dynamic EOQ modeling, supplier lead-time evaluation, and automated 3-way invoice matching.
- **Input Triggers:** `erp.inventory.stock_level_changed`, `erp.supplychain.invoice_received`.

- **Read Access:** stock_ledger_entries, purchase_orders, goods_receipt_notes, supplier_scorecards.
- **Write Access:** purchase_orders, purchase_requisitions, material_quarantine_flags.
- **MCP Tool Bindings:** extract_invoice_multimodal, execute_three_way_match, calculate_dynamic_rop, submit_purchase_order.

System Prompt: Supply Chain & Procurement Agent

You are the autonomous Supply Chain & Procurement Agent. You ensure material availability while optimizing working capital through stochastic inventory modeling and automated procurement.

Core Operating Invariants:

1. Disregard static reorder levels. Continually calculate Dynamic Reorder Points (ROP) utilizing lead-time variance, consumption velocity, and target service levels ($Z=2.33$ for 99% availability).
2. Execute automated 3-way matching across Invoices, Purchase Orders (POs), and Goods Receipt Notes (GRNs). Enforce tolerances: SKU must match 100%, quantity must be $\leq$ received quantity, unit price variance must be $\leq 1.0\%$.
3. If 3-way matching passes within tolerance, stage the AP invoice for payment. If matching fails, generate an itemized dispute artifact and notify the supplier.
4. Continuously evaluate supplier on-time-in-full (OTIF) metrics. When supplier lead-time variance increases by more than 15%, automatically expand the safety stock buffer.

Production & Shop-Floor Agent

- **Operational Role:** Manages real-time job-shop scheduling, IoT machine condition monitoring, predictive maintenance dispatching, and dynamic workstation re-routing.
- **Input Triggers:** erp.production.iot_telemetry, erp.production.machine_fault.
- **Read Access:** work_orders, workstation_capacities, machine_sensor_logs, operator_certifications.
- **Write Access:** workstation_dispatch_queue, maintenance_tickets, work_order_routings.
- **MCP Tool Bindings:** solve_job_shop_schedule, isolate_workstation, dispatch_maintenance_order, read_iot_telemetry.

System Prompt: Production & Shop-Floor Agent

You are the autonomous Production & Shop-Floor Agent. You maintain manufacturing throughput and operational equipment effectiveness (OEE) across all shop-floor assets.

Core Operating Invariants:

1. Model factory operations as a flexible job-shop scheduling problem. Formulate in

n_134](end_span)terval variables, precedence relations, and machine non-overlap constraints via the OR-Tools CP-SAT solver service.

2. Ingest IoT sensor telemetry (vibration, heat, power consumption). If predictive models indicate imminent mechanical failure ($P_{\text{failure}} > 0.85$ within 48 hours), automatically isolate the `ass[span_123](start_span)[span_123](end_span)[span_135](start_span)[span_135](end_span)`et and generate a maintenance work order.
3. Upon receiving machine failure telemetry, re-solve the factory schedule within 15 seconds. Re-route jobs to qualified alternative workstations while minimizing makespan disruption.
4. Never schedule an operation on a machine unless the assigned operator holds an active, verified safety certification.

Workforce (HR) Agent

- **Operational Role:** Handles shift assignments, automated shift-swapping, labor law compliance monitoring, and multi-modal travel expense validation.
- **Input Triggers:** `erp.workforce.swap_requested`, `erp.workforce.expense_submitted`.
- **Read Access:** `employee_profiles`, `shift_rosters`, `labor_regulations`, `expense_claims`.
- **Write Access:** `confirmed_schedules`, `expense_approvals`, `compliance_infractions`.
- **MCP Tool Bindings:** `verify_labor_compliance`, `execute_shift_trade`, `audit_receipt_multimodal`, `authorize_expense_payout`.

System Prompt: Workforce

(HR) [span_273](start_span)[span_273](end_span) Agent

You are the autonomous Workforce Agent. You ensure factory and corporate operational readiness while enforcing labor regulations and corporate expense policies.

Core Operating Invariants:

1. Strictly enforce all statutory labor laws and union contracts. Enforce mandatory rest intervals (minimum 11 consecutive hours between shifts) and maximum weekly work hours (48 hours rolling).
2. Autonomously process shift-swap requests. Verify skill certifications, overtime thresholds, and labor law boundaries. If valid, commit the schedule change in under 30 seconds without supervisory delay.
3. Parse employee expense receipts using multi-modal extraction. Cross-check merchant categories, date alignments, per-diem caps, and receipt image uniqueness to prevent duplicate reimbursement.
4. Automatically authorize valid expense reports under 500 USD that satisfy all corporate travel policies.

Revenue & CRM Agent

- **Operational Role:** Ingests inbound sales requests, checks available-to-promise inventory,

- calculates margin-defended dynamic pricing, and issues customer quotations.
- **Input Triggers:** `erp.crm.inbound_rfq_email`, `erp.crm.customer_order_placed`.
- **Read Access:** `product_catalog`, `inventory_available_to_promise`, `cost_structures`, `customer_history`.
- **Write Access:** `sales_quotes`, `customer_orders`, `crm_pipeline_stages`.
- **MCP Tool Bindings:** `parse_rfq_document`, `calculate_landed_margin_price`, `query_stock_atp`, `generate_pdf_quote`.

System Prompt: Revenue & CRM Agent

You are the autonomous Revenue & CRM Agent. You drive commercial sales conversion by generating accurate, margin-defended quotations in response to customer inquiries.

Core Operating Invariants:

1. Ingest unstructured customer RFQ emails and documents via B[`span_195`] (`start_span`) [`span_195`] (`end_span`) AML parsing. Extract target SKUs, quantities, and delivery deadlines.
2. Query Available-to-Promise (ATP) inventory and production capacity to verify realistic delivery schedules before offering delivery commitments.
3. Compute dynamic pricing using real-time landed component costs, plant operating overhead, and dynamic freight indices. Maintain a [`span_152`] (`start_span`) [`span_152`] (`end_span`) strict minimum contribution margin floor (default: 22%).
4. Compile professional, legally bound PDF quotations and dispatch them to prospective clients within 5 minutes of initial inbound inquiry.

Compliance & Guardrail Agent

- **Operational Role:** Operates as a horizontal auditor and transaction gatekeeper, verifying enterprise policy, RBAC boundaries, and regulatory compliance before state commits.
- **Input Triggers:** `erp.system.transaction_commit_requested` (Synchronous verification pipeline).
- **Read Access:** All enterprise data tables, regulatory statutes, historical audit logs, access matrices.
- **Write Access:** `audit_verification_tokens`, `system_quarantine_locks`.
- **MCP Tool Bindings:** `evaluate_statutory_rules`, `verify_agent_rbac`, `sign_audit_block`, `reject_transaction_mutation`.

System Prompt: Compliance & Guardrail Agent

You are the horizontal Compliance & Guardrail Agent. You serve as the final verification firewall of the ERP system, ensuring zero legal, regulat[`span_252`] (`start_span`) [`span_252`] (`end_span`)ory, or policy infractions occur.

Core Operating Invariants:

1. Intercept every state-changing transactional mutation proposed by any agent across the enterprise mesh.
2. Evaluate the proposed transaction against the enterprise Role-Based Access Control (RBAC) matrix and the deterministic statutory rules

engine (SOX, GAAP, IFRS, OSHA, GDPR).

3. If an action violates any hard business rule or statutory constraint, issue an immediate, non-bypassable rejection veto accompanied by an immutable audit rationale.

4. Calculate a cryptographic SHA-256 hash linking the decision context (prompt, model version, retrieved documents, policy rules) and commit the verification record to the immutable audit ledger.

Functional Epics & User Stories

The transition to an AI-Native ERP transforms ten foundational operational workflows from batch-oriented, human-driven forms into autonomous event streams.

Accounts Payable: Automated 3-Way Matching and Variance Routing

- **Legacy Process:** Accounts payable clerks manually open paper/PDF invoices, verify numbers against purchase orders in ERPNext, check warehouse delivery receipts, and route approval requests via email.
- **AI-Native Workflow:** Inbound invoice documents are parsed via BAML multi-modal models, matched against purchase orders and goods receipt notes in the transactional database, and committed autonomously if variances sit within pre-set limits.
- **Acceptance Criteria:**
 1. System ingests PDF/image invoices via email webhook, achieving line-item extraction with confidence score ≥ 0.95 in under 3.0 seconds.
 2. Software validates 3-way matching invariants: SKU identity match = 100%, $Quantity_{\text{Invoice}} \leq Quantity_{\text{GRN}}$, and $\frac{Price_{\text{Invoice}}}{Price_{\text{PO}}} \leq 0.010$ (1.0% price tolerance).
 3. If tolerances are met, an AP journal entry is staged and committed to the general ledger with zero manual touchpoints.
 4. If tolerances are breached, system generates an itemized discrepancy notice, posts it to the vendor portal, and sets document status to DISPUTED.

Inventory Replenishment: Predictive Demand Modeling

- **Legacy Process:** Inventory managers check static Min/Max reorder points once a month, cross-referencing warehouse counts against past sales trends in manual spreadsheets.
- **AI-Native Workflow:** Stock consumption events trigger dynamic reorder calculations incorporating lead-time distributions, rolling sales velocity, and seasonal variance to generate automated purchase orders.
- **Acceptance Criteria:**
 1. Every stock_consumed event updates the rolling daily demand distribution (μ_D, σ_D) and lead-time distribution (μ_L, σ_L) in real time.
 2. Dynamic Reorder Point is calculated continuously: $ROP = \mu_D + Z \sqrt{\mu_L \sigma_D^2 + \mu_D^2 \sigma_L^2}$ enforcing $Z = 2.33$ (99% service

- availability).
3. When $Stock_{\text{On-Hand}} + Stock_{\text{On-Order}} \leq ROP$, system generates a dynamic PO using the Economic Order Quantity (EOQ) formula.
 4. POs under 25,000 USD are signed and transmitted directly to the vendor via EDI/API in under 60 seconds without manual procurement intervention.

Quotation Generation: Unstructured RFQ-to-PDF Pipeline

- **Legacy Process:** Sales representatives read inbound inquiries, check stock levels in ERP forms, phone shop-floor managers for build schedules, and draft PDF quotes manually over 24 to 72 hours.
- **AI-Native Workflow:** Inbound customer emails are parsed for technical requirements, stock availability-to-promise is verified, dynamic margin-defended pricing is calculated, and an official PDF quote is returned within minutes.
- **Acceptance Criteria:**
 1. Inbound customer emails with attachments are parsed by the Revenue Agent via BAML in under 5 seconds, extracting SKUs, specifications, and volume tiers.
 2. System queries real-time inventory and scheduled production runs to compute Available-to-Promise (ATP) delivery dates with 98% accuracy.
 3. Quote price dynamically computes component costs, labor, and machine depreciation, enforcing a minimum contribution margin floor of 22%.
 4. Completed PDF quote is compiled and delivered via email to the client within 300 seconds of initial receipt.

Production Routing: Dynamic Constraint-Based Rescheduling

- **Legacy Process:** Work orders follow static sequential workstation routings; if a critical machine fails, lines stall while supervisors spend hours manually updating assignments.
- **AI-Native Workflow:** Edge IoT failure telemetry triggers constraint programming models (Google OR-Tools CP-SAT), re-routing queued operations across qualified alternative machinery with minimal makespan disruption.
- **Acceptance Criteria:**
 1. Machine fault telemetry (status = CRITICAL_FAULT) triggers a rescheduling event within 100 milliseconds.
 2. The Production Agent launches an OR-Tools CP-SAT solver defining interval variables for all unstarted operations, asserting machine capability constraints and NoOverlap execution rules.
 3. The solver computes a global makespan-minimized schedule in under 15 seconds.
 4. Digital work order routings update in PostgreSQL, pushing revised job instructions to operator floor displays immediately.

Bank Reconciliation: Continuous Open-Banking Feed Matching

- **Legacy Process:** Treasury staff download monthly CSV statements, manually matching rows against invoices and clearing ledger accounts over several days during month-end closes.
- **AI-Native Workflow:** Live open-banking feeds stream transaction data continuously,

matching journal entries via vector embeddings and fuzzy logic to close the ledger in real time.

- **Acceptance Criteria:**
 1. Open-banking webhooks ingest settlement records in real time.
 2. System combines exact amount and date filtering with pgvector cosine similarity over counterparty names and invoice text.
 3. Transactions matching with confidence ≥ 0.92 post automated clearing entries (Debit Bank, Credit Accounts Receivable) within 5 seconds.
 4. Ambiguous transactions (confidence < 0.92) are placed into an accountant reconciliation worklist with the top three candidate matches ranked by semantic score.

Expense Management: Multi-Modal Receipt Auto-Approvals

- **Legacy Process:** Employees staple paper receipts to monthly forms, passing through multi-tiered managerial approval queues that take weeks to reimburse routine travel expenses.
- **AI-Native Workflow:** Mobile receipt uploads are analyzed via vision models, checked against corporate travel limits and tax rules, and approved for payout within 60 seconds.
- **Acceptance Criteria:**
 1. Mobile camera images parse through multi-modal BAML functions in under 4 seconds, extracting vendor name, tax ID, itemized totals, currency, and date.
 2. Expense rules are evaluated deterministically: meal caps (\$75 USD/day), hotel limits, alcohol restrictions, and image hash deduplication.
 3. Fully compliant claims under 500 USD commit an automated AP reimbursement record without human review.
 4. Non-compliant claims are flagged, generating a contextual notification to the employee explaining the policy violation.

Quality Control: Edge Vision Defect Detection and Quarantine

- **Legacy Process:** Quality staff manually inspect a small sample of manufactured parts at the end of shifts, resulting in delayed discovery of tool wear and defective production lots.
- **AI-Native Workflow:** Industrial edge cameras execute real-time visual inspection, automatically triggering physical scrap diversion and inventory lot quarantine upon defect identification.
- **Acceptance Criteria:**
 1. Edge vision models evaluate production item images, publishing defect classifications (e.g., surface crack, dimensional variance) to Redpanda in under 50 milliseconds.
 2. Upon receiving defect confidence > 0.98 , the system issues a hardware PLC trip command diverting the defective item in under 200 milliseconds.
 3. Compliance Agent flags the associated production lot as QUARANTINED in the stock ledger, blocking warehouse picking and shipping.
 4. If defect rates breach 3.0% over a rolling 1-hour window, the system pauses upstream workstation operations and notifies maintenance technicians.

Shift Scheduling: Automated Compliance-Checked Shift Swapping

- **Legacy Process:** Factory workers trade shifts using physical paper slips, requiring shift supervisors to manually verify overtime rules, union agreements, and operator skills.
- **AI-Native Workflow:** Workers initiate trades in a mobile interface; the Workforce Agent verifies certifications, labor laws, and overtime caps, approving changes in seconds.
- **Acceptance Criteria:**
 1. Peer-to-peer shift trade requests trigger statutory and enterprise constraint checks within 500 milliseconds.
 2. The system verifies mandatory labor invariants: ≥ 11 hours of continuous rest between shifts, rolling 7-day cumulative hours ≤ 48 , and matching machine operation certifications.
 3. Valid trades update the master schedule roster in PostgreSQL and push confirmation messages to both workers in under 30 seconds.
 4. Non-compliant trades are rejected with an itemized explanation of the specific labor rule or safety certification constraint violated.

Equipment Maintenance: Sensor-Driven Predictive Work Orders

- **Legacy Process:** Maintenance follows static calendar intervals (e.g., servicing motors every 90 days), resulting in unnecessary maintenance or catastrophic unpredicted breakdowns.
- **AI-Native Workflow:** Continuous IoT telemetry streams into anomaly detection pipelines, automatically generating work orders and ordering parts before machine failure occurs.
- **Acceptance Criteria:**
 1. Machine sensors stream vibration RMS, bearing temperature, and motor power to Redpanda every 100 milliseconds.
 2. Anomaly models detect component degradation when metrics exceed baseline thresholds (Vibration $> 4.5 \text{ mm/s}$ or $\Delta T > 2.0^\circ \text{C/hr}$ over 6 hours).
 3. When failure probability exceeds 0.85 within a 48-hour horizon, the Production Agent automatically creates a maintenance work order.
 4. The system checks spare part inventory for required replacement assemblies; if stock is absent, it generates an expedited purchase order.

Pricing Strategy: Real-Time Landed Cost Margin Protection

- **Legacy Process:** Price sheets are updated annually in static databases; sudden increases in raw material or logistics costs degrade product gross margins unnoticed.
- **AI-Native Workflow:** Material price spikes detected on inbound invoices trigger real-time Bill of Materials (BOM) cost updates and automatically adjust quote price floors.
- **Acceptance Criteria:**
 1. Inbound supplier invoices that introduce purchase price deltas trigger an automated recalculation of parent assembly landed costs within 5 seconds.
 2. The system identifies all active, unaccepted sales quotations containing the affected items.
 3. If landed cost increases depress contribution margin below the corporate threshold

- (22%), the system invalidates the unaccepted quote.
4. The Revenue Agent generates an updated quotation reflecting current costs and dispatches an explanatory price adjustment notice to the customer.

Guardrails & Security Posture

Deploying autonomous agents into operational enterprise workflows requires strict, multi-layered security controls. The architecture implements a defense-in-depth model that prevents probabilistic AI models from writing unvalidated mutations directly to system ledgers.

Deterministic Ledger Firewall and Invariant Validation Pipeline

Stage 1: Agent Reasoning & Proposal Generation

- Domain subagent formulates structured mutation proposal via BAML.
- Submits candidate action via MCP JSON-RPC to the Deterministic Ledger Engine.

Stage 2: Deterministic Rule & Invariant Evaluation

- Invariant Check 1: Zero-Sum Balance: $\text{SUM}(\text{Debits}) - \text{SUM}(\text{Credits}) == 0.0000$.
- Invariant Check 2: Account Code Validation: Active in verified Chart of Accounts.
- Invariant Check 3: Accounting Period Lock: Posting date falls within open fiscal period.
- Invariant Check 4: Monetary Autonomy Ceilings: Verified against transaction limits.

Stage 3: Decision Branching & Commitment

- If any invariant fails:
 - * Immediate transaction abort
 - `a[span_159] (start_span) [span_159] (end_span)n[span_59] (start_span) [span_59] (end_span)d rollback.`
 - * Failure event emitted to Redpanda; security alert logged to `agent_audit_logs`.
- If all invariants pass:
 - * PostgreSQL ACID
 - `t[span_160] (start_span) [span_160] (end_span)ransaction commits changes to general_ledger_entries.`
 - `[span_46] (start_span) [span_46] (end_span) [span_55] (start_span) [span_55] (end_span) * Transactional outbox event inserted; Debezium CDC streams update.`
 - * Cryptographic SHA-256 hash appended to immutable audit chain.

Deterministic Financial Invariants and Rule Enforcement

Every state mutation that impacts general ledger balances, inventory balances, or compliance rosters must pass through the Deterministic Ledger Engine. Written in Rust and deployed as an

isolated service, this engine evaluates four non-negotiable operational invariants:

1. **Zero-Sum Ledger Balance Invariant:** $\sum_{i=1}^n \text{Debit}_i - \sum_{i=1}^n \text{Credit}_i = 0.0000$ Floating-point arithmetic is strictly prohibited; all currency amounts use arbitrary-precision fixed-point math (NUMERIC(18, 4)).
2. **Fiscal Period Locking:** Transactions with posting dates in closed fiscal months or years are rejected immediately, regardless of agent confidence scores or supervisory prompts.
3. **Chart of Accounts Integrity:** All debit and credit entries must reference active, valid account codes and matching cost-center dimensions defined in the corporate chart of accounts.
4. **Autonomous Financial Ceilings:** Agent autonomy is restricted by hard-coded financial tiers:
 - *Tier 1 (Sub-2,500 USD):* Fully autonomous processing; instant commit.
 - *Tier 2 (2,500 to 25,000 USD):* Autonomous processing with automated supervisory notification.
 - *Tier 3 (Over 25,000 USD):* Staged mutation requiring cryptographic human executive approval prior to ledger commit.

Immutable Audit Logging and Cryptographic Lineage

To satisfy external financial and regulatory audits (SOX, SOC 1/2, ISO 27001, GAAP, IFRS), the system logs every autonomous operational decision in the append-only `agent_audit_logs` table. Each record stores:

- **Distributed Trace ID:** Standardized OpenTelemetry trace identifier linking the initial edge event, orchestrator decomposition, subagent reasoning steps, and database mutations.
- **Model Provenance:** The model provider, exact release checkpoint (e.g., `gemini-3.1-pro-202602`), temperature setting (0.0), and the git-commit hash of the BAML prompt template.
- **Context Hashes:** Cryptographic SHA-256 hashes of all semantic text chunks, policy documents, and database rows retrieved during execution.
- **Deterministic Rule Assertions:** A JSON snapshot documenting every compliance constraint, accounting invariant, and RBAC permission verified during evaluation.
- **Cryptographic Hash Chain:** Every audit row includes a SHA-256 block hash calculated from its own content concatenated with the hash of the preceding audit record, providing tamper evidence across the entire audit trail.

Role-Based Access Control and Execution Boundaries

Agents authenticate using short-lived (15-minute) JSON Web Tokens (JWTs) issued by Supabase Auth. PostgreSQL Row-Level Security (RLS) policies enforce least-privilege data access across all database tables.

```
-- Production Row-Level Security Configuration
ALTER TABLE general_ledger_entries ENABLE ROW LEVEL SECURITY;
ALTER[span_258](start_span)[span_258](end_span) TABLE
stock_ledger_entries ENABLE ROW LEVEL SECURITY;

-- Read Policy: Domain agents can read only their authorized tables
CREATE POLICY agent_gl_read_policy ON general_ledger_entries
```

```

FOR SELECT
TO authent[span_253] (start_span) [span_253] (end_span) icated
USING (
    auth.jwt() ->> 'agent_role' IN ('FINANCIAL_CONTROLLER',
'COMPLIANCE_GUARDRAIL', 'CHIEF_ORCHESTRATOR')
    AND
tena[span_173] (start_span) [span_173] (end_span) [span_176] (start_span) [s
pan_176] (end_span)nt_id = (auth.jwt() ->> 'tenant_id')::UUID
);

-- Write Policy: BANS direct agent insertion into financial tables
-- All mutations must be executed by the internal Deterministic Ledger
Engine service role
CREATE POLICY ledger_engine_exclusive_write ON general_ledger_entries
    FOR INSERT
    TO authenticated
    WITH CHECK (

auth.jwt() [span_47] (start_span) [span_47] (end_span) [span_56] (start_span
) [span_56] (end_span) ->> 'service_role' = 'ledger_engine_internal'
);

```

Implementation Roadmap

The engineering roadmap utilizes Google Antigravity to accelerate software delivery. By leveraging Antigravity's multi-agent development manager, parallel subagent code-generation capabilities, and automated verification loops, the system will be deployed across four phased milestones over a twelve-month delivery window.

Phased Engineering Roadmap Milestones

Phase 1: Foundation & Data Substrate (Months 1-3)

- Core PostgreSQL database schemas and constraints
- Transactional outbox CDC streaming via Debezium and Redpanda
- Deterministic Ledger Engine validation service in Rust
- Semantic vector store indexing via pgvector

Phase 2: Core Orchestration & Financial AP (Months 4-6)

- Chief Orchestrator and Compliance Agent implementation via Ant[span_390] (start_span) [span_390] (end_span) [span_391] (start_span) [span_391] (end_span)igravity SDK
- BAML prompt engineering contracts and type-generation pipelines
- Automated Accounts Payable and live open-banking reconciliati[span_392] (start_span) [span_392] (end_span) [span_394] (start_span) [span_394] (end_span)on pipelines
- Next.js administrative dashboard and Antigravity artifac[span_396] (start_span) [span_396] (end_span)t visualizer

Phase 3: Dynamic Supply Chain & Shop Floor (Months 7-9)

- Supply Chain & Procurement Agent with dynamic replenishment modeling
- Production & Shop-Floor Agent with Google OR-Tools CP-SAT scheduler
- IoT edge sensor ingestion pipelines and predictive maintenance triggers
- Machine telemetry event handlers and dynamic re-routing logic

Phase 4: Full Autonomous Agent Mesh (Months 10-12)

- Workforce (HR) Agent with automated shift swapping and receipt auditing
- Revenue & CRM Agent with margin-defended dynamic pricing
- Priority-based conflict resolution engine and cryptographic audit chain
- Enterprise load testing, disaster recovery and SOC 2 Type II certification

Engineering Delivery Matrix

Implementation Phase	Core Deliverables	Stack & Tooling	Antigravity Development Modality	Verification Gate & Exit Criteria
Phase 1: Foundation & Substrate (Months 1–3)	• PostgreSQL relational schema definitions. • Transactional Outbox pattern with Debezium CDC. • Redpanda cluster setup with SASL/SCRAM security. • Deterministic Ledger Engine service in Rust.	• PostgreSQL / Supabase • Debezium CDC • Redpanda broker • Rust / Python services	• Spawn parallel Antigravity coding subagents for migration script generation. • Antigravity task tracking for database schema design. • Automated unit test generation via IDE.	• Zero dual-write data inconsistencies under 10,000 tx/sec. • Enforce zero-sum invariant across 10 ⁶ synthetic transactions. • CDC outbox streaming latency < 20 milliseconds.
Phase 2: Orchestration & Finance (Months 4–6)	• Chief Orchestrator & Compliance Guardrail Agents. • BAML structured	• Antigravity Python SDK • BAML compiler • Model Context Protocol • Next.js /	• Antigravity Implementation Plans for MCP server scaffolding. • Visual diff	• AP invoice line-item extraction confidence > 0.95. • 90% touchless 3-way

Implementation Phase	Core Deliverables	Stack & Tooling	Antigravity Development Modality	Verification Gate & Exit Criteria
	prompt-to-code pipelines. • Model Context Protocol (MCP) server integration. • Accounts Payable & Open-Banking pipelines.	Tailwind CSS	reviews of BAML type generators. • Browser subagents for end-to-end web UI verification.	matching across POs and GRNs. • Continuous bank reconciliation accuracy ≥ 98%.
Phase 3: Supply Chain & Factory (Months 7–9)	• Supply Chain & Procurement Agent deployment. • Production & Shop-Floor Agent deployment. • Google OR-Tools CP-SAT scheduler service. • IoT edge telemetry streaming broker.	• Google OR-Tools • MQTT / OPC-UA bridge • pgvector semantic store • Redis distributed cache	• Parallel Antigravity subagents formulating CP-SAT constraint models. • Review visual artifacts of job-shop schedule graphs. • Antigravity CLI headless integration runs.	• Dynamic job-shop rescheduling completes in ≤ 15 seconds. • Predictive maintenance alerts trigger 48 hours prior to fault. • Eliminate production inventory stockouts.
Phase 4: Full Agent Mesh (Months 10–12)	• Workforce (HR) Agent deployment. • Revenue & CRM Dynamic Pricing Agent. • Mathematical priority conflict engine. • Immutable cryptographic audit block chaining.	• Gemini 3.1 Pro / Flash • Complete Agent Mesh • Next.js Enterprise UI • OpenTelemetry tracing	• Antigravity Agent Manager panel tracking multi-agent teamwork. • End-to-end regression suites running across isolated worktrees. • Automated deployment via CLI.	• Priority-based conflict resolution passes all edge cases. • Inbound RFQ-to-quote latency ≤ 5 minutes. • Complete third-party SOC 2 Type II compliance audit.

Transitioning from monolithic, form-driven ERP architectures to an autonomous multi-agent system replaces batch-oriented manual processes with continuous, real-time enterprise intelligence. Combining an asynchronous event substrate, strongly typed prompt interfaces, and combinatorial constraint solvers allows the platform to react to operational disruptions in milliseconds. Crucially, by enforcing deterministic validation rules at the ledger interface, the system prevents probabilistic language models from compromising financial records. The result is an adaptive, auditable enterprise operating system that resolves cross-departmental friction autonomously while preserving mathematical and regulatory integrity.

Works cited

1. From Traditional ERP to Agentic ERP - Medium, <https://medium.com/@jannadikhemais/from-traditional-erp-to-agentic-erp-afc67ea88e4c> 2. Implementing Data Partitioning, Long-Term Archiving & Legacy Data, <https://clefincode.com/blog/global-digital-vibes/en/implementing-data-partitioning-long-term-archiving-legacy-data-access-in-erpnext> 3. What Is Agentic ERP? | AI Agents in ERP - CLaaS2SaaS, <https://claas2saas.com/blog/what-is-agentic-erp/> 4. Deadlock issue on Frappe v14 - ERPNext, <https://discuss.frappe.io/t/deadlock-issue-on-frappe-v14/109049> 5. The Outbox Pattern Explained: Reliable Event Publishing for, <https://streamkap.com/resources-and-guides/outbox-pattern-explained> 6. Transactional Outbox: Database-Kafka Consistency - Conduktor, <https://www.conduktor.io/blog/transactional-outbox-pattern-database-kafka> 7. How ERP is evolving in the agentic AI era | Deloitte US, <https://www.deloitte.com/us/en/what-we-do/capabilities/applied-artificial-intelligence/articles/how-erp-is-evolving-in-agentic-ai-era.html> 8. Multi-Agent Large Language Model Architecture for Autonomous, <https://arxiv.org/html/2607.17331v1> 9. Google Antigravity - Wikipedia, https://en.wikipedia.org/wiki/Google_Antigravity 10. Priority-Based Conflict Resolution - Emergent Mind, <https://www.emergentmind.com/topics/priority-based-conflict-resolution> 11. MCP Use Cases: Real-World Applications for AI Agents | Blaxel Blog, <https://blaxel.ai/blog/mcp-use-cases> 12. Subagents | Google Antigravity Docs, <https://antigravity.google/docs/subagents/> 13. Multi-Agent Conflict Resolution Fram - SARC Publisher, <https://sarcouncil.com/download-article/SJECS-525-2025-142-152.pdf> 14. (PDF) MultiLevel Conflict in Multi-Agent Systems - ResearchGate, https://www.researchgate.net/publication/245246758_MultiLevel_Conflict_in_Multi-Agent_Systems 15. BAML: The Structured-Output Power Tool Your LLM Workflow Has, <https://medium.com/@manavisrani07/baml-the-structured-output-power-tool-your-llm-workflow-has-been-missing-f326046d019b> 16. Subagents, Hooks, Scheduled Tasks, Agent Management, Voice, <https://antigravity.google/blog/google-io-2026-feature-deep-dive> 17. Artifacts - Google Antigravity, <https://antigravity.google/docs/artifacts/> 18. [Video] Outbox meets change data capture (feat. .NET, PostgreSQL, <https://dev.to/joaoofbantunes/video-outbox-meets-change-data-capture-feat-net-postgresql-kafka-and-debezium-1kpl> 19. Getting Started with Google Antigravity - Codelabs, <https://codelabs.developers.google.com/getting-started-google-antigravity> 20. BAML vs Instructor: Structured LLM Outputs - Rost Glukhov, <https://www.glukhov.org/llm-performance/benchmarks/baml-vs-instruct-for-structured-output-llm-in-python/> 21. Unleashing the Power of BAML in LLM Applications, <https://thedataexchange.media/baml/> 22. Schema Reference - What is the Model Context Protocol (MCP)?, <https://modelcontextprotocol.io/specification/2025-11-25/schema> 23. Model Context Protocol: An essential standard for AI-powered tool, <https://retool.com/blog/what-is-model-context-protocol> 24. Building Effective AI Agents: Architecture Patterns and ... - Anthropic, <https://resources.anthropic.com/hubfs/Building%20Effective%20AI%20Agents-%20Architecture%20Patterns%20and%20Implementation%20Frameworks.pdf> 25. Priority-Driven Hierarchical Multi-Agent Systems with Fine-Tuned, <https://www.mdpi.com/2076-3417/16/16/8250> 26. Model Context Protocol (MCP): A Practical Guide for AI Agent, <https://medium.com/@ryagoel1994/model-context-protocol-mcp-a-practical-guide-for-ai-agent-developers-4325c1b33062> 27. NetSuite Bank Reconciliation: Auto-Match Rules & Workflows,

<https://www.houseblend.io/articles/netsuite-bank-reconciliation-auto-match-rules> 28. A Comprehensive Benchmark of Constraint Programming Solvers,
<https://www.preprints.org/manuscript/202605.1319> 29. Complex constraints using OR Tools in Python for a scheduling,
<https://stackoverflow.com/questions/59437151/complex-constraints-using-or-tools-in-python-for-a-scheduling-problem> 30. Agentic ERP Architecture: Designing AI-Driven Enterprise Systems,
<https://erpsoftwareblog.com/2026/04/agentic-erp-architecture-designing-ai-framework/> 31. OR-TOOLS Job Shop Scheduling - splitting longer tasks and,
<https://stackoverflow.com/questions/78291694/or-tools-job-shop-scheduling-splitting-longer-task-s-and-keeping-them-together> 32. Articsledge AI Accounting Software: Features & Pricing 2026,
<https://www.articsledge.com/post/articsledge-ai-accounting-software> 33. The Application of Machine Learning in Developing Next-generation,
<https://research-repository.rmit.edu.au/ndownloader/files/50764632> 34. Data Reconciliation with GenAI - Shashank Guda,
<https://shashankguda.medium.com/data-reconciliation-with-genai-de7e4cd707da> 35. Integrating Oracle Fusion ERP, SADAD, Mada - IRE Journals,
<https://www.irejournals.com/formatedpaper/1722966.pdf>